

Laboratório de Concorrência · Aula 10

Natália Possani de Lima · RA 24190450-2 · Turma esoft6s-c

Gerado em 09/10/2026, 20:10:19 · tempo registrado no cronômetro: 50:52

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

a diferença entre sincronização de cooperação e de competição é que enquanto uma busca o trabalho em equipe para alcançar um objetivo em comum, a outra busca a disputa do mesmo recurso exclusivo

exemplos:

cooperação

- Uma pessoa bate a massa e a outra coloca o bolo no forno. A segunda pessoa não pode colocar o bolo no forno até que a primeira termine de bater a massa. Elas trabalham em equipe para um mesmo objetivo

competição

- banheiro: A tranca da porta do banheiro serve como o sincronizador. Quando alguém entra e tranca a porta, o recurso fica "bloqueado". Quem está de fora precisa esperar na fila até que a tranca mude para "livre"

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

2000000

1ª execução (n = 1.000.000)

1017141

2ª execução (n = 1.000.000)

1656335

3ª execução (n = 1.000.000)

1002528

Saída do programa

não respondido



The screenshot shows an IDE with a file named 'Main.java'. The code is a Java program demonstrating thread synchronization. It defines a shared static variable 'contador' and a 'Runnable' interface implementation 'incrementa'. Two threads, 't1' and 't2', are created and started. The program prints the expected value (2000000) and the actual value (1002528). The console output on the right shows 'esperado: 2000000' and 'obtido: 1002528'.

```
1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1_000_000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
```

Console Output:

```
esperado: 2000000
obtido: 1002528
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

os valores não são os esperados e não se repetem. a execução muda de uma para a outra porque a ordem em que as threads são executadas depende inteiramente do escalonador de processos do sistema operacional. como fatores externos afetam a troca de contexto entre as threads, o momento exato em que cada uma lê e escreve na memória muda a cada execução

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

1. a thread 1 lê o valor 10 para o seu registrador. antes de continuar, ela é pausada pelo sistema operacional
2. a thread 2 assume o controle da CPU. ela lê o valor 10 (pois a Thread 1 ainda não atualizou a memória), soma 1 e grava 11 de volta no contador
3. a thread 1 volta a executar. ela usa o valor antigo que estava no seu registrador (10), soma 1 e grava 11 de volta no contador

1.3 Com $n = 1000$, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

o programa exibe o valor exato esperado. já que o n é muito pequeno, a execução de cada thread é extremamente rápida e termina antes que o sistema operacional faça uma troca de contexto entre elas

o risco que esse tipo de erro proporciona:

- falso positivo: como o erro não aparece com cargas baixas, os testes automatizados e o ambiente de desenvolvimento podem indicar que o código está perfeito

como se comportaria:

- falha catastrófica em produção: em um ambiente real, com servidores sob alta carga, múltiplos núcleos de CPU e milhões de requisições simultâneas, as condições de corrida ocorrem inevitavelmente. isso gera corrupção de dados silenciosa que são extremamente difíceis de reproduzir, rastrear e corrigir

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8

ABABAB → 6

AABABB → 6

AABBAB → 6

BBBAAA → 7

BABABA → 4

AABBBA → 4

Saída do programa

AAABBB → 8

BBBAAA → 7

ABABAB → 6

Ordens possíveis: 20

TOTAL = 4: 9 ordens, por exemplo AABBBBA

TOTAL = 6: 9 ordens, por exemplo AABABB

TOTAL = 7: 1 ordens, por exemplo BBBAAA

TOTAL = 8: 1 ordens, por exemplo AAABBB

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

20 ordens possíveis, 4 ordens diferentes. somente uma ordem dá o resultado correto

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Passo a Passo da Execução AABBBBA:

- Início: TOTAL = 3

1. A (Passo 1 de A): A tarefa A lê o TOTAL atual (3) e guarda em sua memória local.

2. A (Passo 2 de A): A tarefa A calcula o seu próximo valor baseado no que leu: $(3 + 1 = 4)$.

3. B (Passo 1 de B): A tarefa B é intercalada. Ela lê o TOTAL atual, que ainda é 3 (porque A calculou, mas ainda não gravou).

4. B (Passo 2 de B): A tarefa B calcula o seu próximo valor baseado no que leu: $(3 + 2 = 5)$.

5. B (Passo 3 de B): A tarefa B grava o valor calculado no TOTAL. Agora, TOTAL = 5.

6. A (Passo 3 de A): A tarefa A volta a executar e grava o seu valor calculado lá no passo 2. Ela grava 4 no TOTAL, sobrescrevendo o 5 que B tinha acabado de colocar. Agora, TOTAL = 4.

O que aconteceu com a operação de B?

A operação da tarefa B foi completamente perdida (sobrescrita). Como a tarefa A leu o valor inicial antes de B rodar, ela calculou seu resultado com base no valor antigo (3). Quando A finalmente gravou o valor 4, ela apagou o progresso que B havia feito, fazendo com que a computação de B fosse invalidada.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

acredito que a mais rápida será ou a operação atômica ou o semáforo, não tenho certeza sobre o monitor

Saída do programa

```
1.
esperado:          20000000
sem sincronizacao: 1026699 em 3 ms
synchronized:      20000000 em 105 ms
AtomicInteger:     20000000 em 23 ms
```

```
2.
esperado:          20000000
sem sincronizacao: 1021013 em 2 ms
synchronized:      20000000 em 85 ms
AtomicInteger:     20000000 em 28 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

em ambos os casos os que deram o valor correto foram o synchronized e o AtomicInteger

A versão sem sincronização é extremamente rápida porque ela não se importa com a consistência. As duas threads acessam e modificam a memória ao mesmo tempo sem nenhuma barreira, atropelando os dados uma da outra.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não, usar um AtomicInteger individual para cada saldo não resolve o problema de uma transferência bancária - se usarmos apenas AtomicInteger, o sistema executa o Passo 1 com sucesso. Mas, se o sistema falhar, cair a energia ou ocorrer um erro exatamente antes de executar o Passo 2, o dinheiro terá sumido da Conta A e nunca chegará à Conta B. O dinheiro simplesmente desaparece do banco

- Para resolver esse problema, você precisa garantir o princípio da Atomicidade no nível da transação (tudo acontece ou nada acontece). São necessárias duas abordagens principais dependendo do cenário

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

acredito que dará erro

Saídas (antes e depois da correção)

não respondido

```

1 // Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
2 // Cole em https://onecompiler.com/java
3 import java.lang.management.ManagementFactory;
4
5 public class Main {
6     static final Object X = new Object(), Y = new Object(); // dois recursos
7
8     static void pausa() {
9         try { Thread.sleep(100); } catch (InterruptedException e) { }
10    }
11
12    public static void main(String[] args) throws Exception {
13        Thread a = new Thread(() -> {
14            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); } }
15        });
16        Thread b = new Thread(() -> {
17            synchronized (X) { pausa(); synchronized (Y) { System.out.println("B terminou"); } }
18        });
19        a.start(); b.start();
20        Thread.sleep(1000); // espera 1 segundo
21        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
22        System.out.println(presas == null ? "sem impasse"
23                           : "IMPASSE: " + presas.length + " threads presas");
24        System.exit(0); // encerra o programa
25    }
26 }

```

Console: A terminou, B terminou, sem impasse

```

1 // Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
2 // Cole em https://onecompiler.com/java
3 import java.lang.management.ManagementFactory;
4
5 public class Main {
6     static final Object X = new Object(), Y = new Object(); // dois recursos
7
8     static void pausa() {
9         try { Thread.sleep(100); } catch (InterruptedException e) { }
10    }
11
12    public static void main(String[] args) throws Exception {
13        Thread a = new Thread(() -> {
14            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); } }
15        });
16        Thread b = new Thread(() -> {
17            synchronized (Y) { pausa(); synchronized (X) { System.out.println("B terminou"); } }
18        });
19        a.start(); b.start();
20        Thread.sleep(1000); // espera 1 segundo
21        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
22        System.out.println(presas == null ? "sem impasse"
23                           : "IMPASSE: " + presas.length + " threads presas");
24        System.exit(0); // encerra o programa
25    }
26 }

```

Console: IMPASSE: 2 threads presas

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

As threads ficaram presas porque ocorreu uma espera circular (Deadlock) provocada pela disputa de recursos em ordens invertidas:

- Thread a: Bloqueou o recurso X e entrou em pausa. Ao acordar, ela tentou bloquear o recurso Y, mas ele já estava ocupado. Ficou esperando permanentemente por Y.
- Thread b: Bloqueou o recurso Y e entrou em pausa. Ao acordar, ela tentou bloquear o recurso X, mas ele já estava ocupado pela Thread a. Ficou esperando permanentemente por X.

Ambas as threads ficaram aguardando que a outra liberasse o respectivo recurso necessário, mas nenhuma delas podia avançar para liberar o seu próprio recurso.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Por que elimina o impasse?

Definir uma ordem estrita (ex: sempre pegar primeiro X e depois Y) quebra a condição de espera circular.

Se a Thread a pegar o recurso X primeiro, a Thread b não conseguirá pegar o recurso X e ficará bloqueada logo no início, antes de conseguir colocar as mãos no recurso Y. Dessa forma, a Thread a consegue obter Y livremente,

termina seu trabalho, libera ambos os recursos e só então a Thread b pode prosseguir.

Exemplo de Sistema Real:

Um exemplo clássico e real ocorre em Transferências Bancárias Simultâneas em um banco de dados:

- O cliente João tenta transferir dinheiro para a Maria (o sistema tenta bloquear a conta do João e depois a da Maria).
 - Exatamente no mesmo milissegundo, a Maria tenta transferir dinheiro para o João (o sistema tenta bloquear a conta da Maria e depois a do João).
- Se o sistema não forçar uma ordenação global (por exemplo, obrigando a travar sempre a conta com o menor número de ID primeiro), as duas transações vão travar o banco de dados mutuamente, gerando um impasse real.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

200ms

Tempo com 1 trabalhador

1 trabalhadores, 6 tarefas de 100 ms: levou 600ms

Tempo com 3 trabalhadores

3 trabalhadores, 6 tarefas de 100 ms: levou 200ms

Tempo com 6 trabalhadores

6 trabalhadores, 6 tarefas de 100 ms: levou 100ms

Saída do programa

não respondido

```
Console I/O AI Agent
trabalhador 2 fez a tarefa 3
trabalhador 3 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 2 fez a tarefa 4
trabalhador 3 fez a tarefa 5
trabalhador 1 fez a tarefa 6
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
```

```
trabalhador 1 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 1 fez a tarefa 5
trabalhador 1 fez a tarefa 6
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms
```

```
trabalhador 2 fez a tarefa 2
trabalhador 1 fez a tarefa 6
trabalhador 6 fez a tarefa 1
trabalhador 3 fez a tarefa 3
trabalhador 4 fez a tarefa 4
trabalhador 5 fez a tarefa 5
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Este programa segue a filosofia clássica do Go: "Não comunique compartilhando memória; compartilhe memória comunicando." Ele dispensa travas explícitas por dois fatores estruturais:

1. Os canais (chan) possuem sincronização embutida:

O canal tarefas gerencia internamente o acesso aos seus dados. Quando múltiplas goroutines (trabalhadores) tentam ler (<-chan) do mesmo canal ao mesmo tempo, o próprio mecanismo do Go garante de forma atômica que uma tarefa seja entregue a apenas um trabalhador, sem que eles colidam ou dupliquem o consumo.

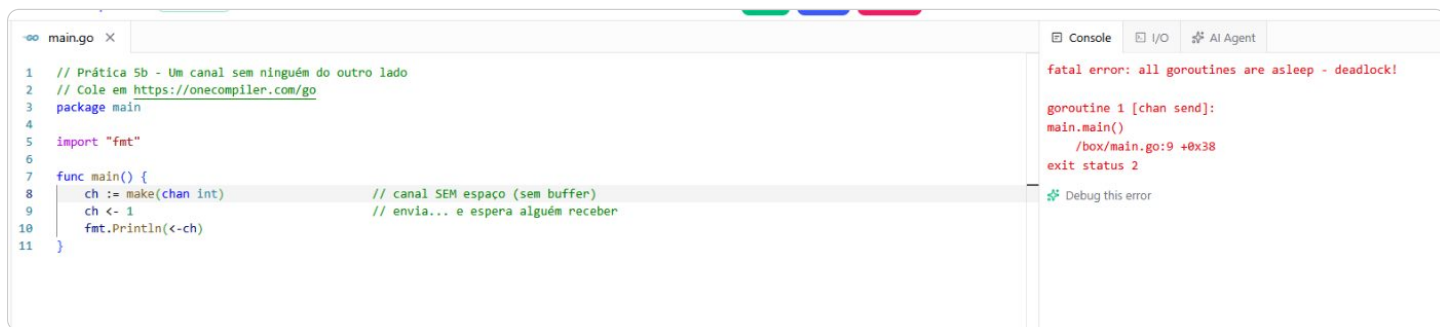
2. Ausência de estado global compartilhado e mutável:

Diferente dos exercícios anteriores (onde várias threads alteravam diretamente a mesma variável TOTAL na memória), os trabalhadores aqui não modificam nenhuma variável compartilhada. Cada um recebe um valor inteiro isolado do canal, processa seu próprio escopo e imprime o resultado na tela. Não há dados disputados, eliminando a possibilidade de condições de corrida.

Saídas (antes e depois da troca)

não respondido

```
main.go x
1 // Prática 5b - Um canal sem ninguém do outro lado
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import "fmt"
6
7 func main() {
8     ch := make(chan int, 1) // canal SEM espaço (sem buffer)
9     ch <- 1 // envia... e espera alguém receber
10    fmt.Println(<-ch)
11 }
```



```
1 // Prática 5b - Um canal sem ninguém do outro lado
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import "fmt"
6
7 func main() {
8     ch := make(chan int)           // canal SEM espaço (sem buffer)
9     ch <- 1                       // envia... e espera alguém receber
10    fmt.Println(<-ch)
11 }
```

fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
/box/main.go:9 +0x38
exit status 2

Debug this error

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O programa trava e resulta em um erro de deadlock (fatal error: all goroutines are asleep - deadlock!) devido à natureza de um canal sem buffer (unbuffered channel).

Ao adicionar o tamanho 1, você está dando um "estoque" ou um espaço de armazenamento temporário para o canal.

- Agora, quando a linha `ch <- 1` é executada, o programa não bloqueia. Ele simplesmente deposita o valor 1 dentro do buffer do canal e prossegue imediatamente para a próxima linha.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece? acredito que dará erro

Saídas (o erro e o resultado com move)

não respondido

```

error[E0373]: closure may outlive the current function, but it borrows `contador`, which is owned by the current function
--> HelloWorld.rs:7:27
7 |   let t = thread::spawn(|| {
  |                         ^^ may outlive borrowed value `contador`
8 |       contador += 1;           // a thread altera a variável de main
  |       ----- `contador` is borrowed here
  |
note: function requires argument type to outlive `'static`
--> HelloWorld.rs:7:13
7 |   let t = thread::spawn(|| {
  |   ^
8 |   |       contador += 1;           // a thread altera a variável de main
9 |   |   });
  |   ^
help: to force the closure to take ownership of `contador` (and any other referenced variables), use the `move` keyword
7 |   let t = thread::spawn(move || {
  |                         ++++

error[E0502]: cannot borrow `contador` as immutable because it is also borrowed as mutable
--> HelloWorld.rs:11:20
7 |   let t = thread::spawn(|| {
  |   -             -- mutable borrow occurs here
  |   |
8 |   |       contador += 1;           // a thread altera a variável de main
  |   |       ----- first borrow occurs due to use of `contador` in closure
9 |   |   });
  |   |   - argument requires that `contador` is borrowed for `'static`
10 |   |   t.join().unwrap();
11 |   |   println!("{}", contador);
  |   |   ^^^^^^^^ immutable borrow occurs here
  |
note: requirement that the value outlives `'static` introduced here
--> /rustc/48a229ceaefd4985c50990b14116b6d856af0985/library/std/src/thread/functions.rs:128:14

error: aborting due to 2 previous errors

Some errors have detailed explanations: E0373, E0502.
For more information about an error, try `rustc --explain E0373`.

```

 [Debug this error](#)

```
warning: value captured by `contador` is never read
--> HelloWorld.rs:8:9
|
8 |         contador += 1;           // a thread altera a variável de main
|         ^^^^^^^^^
= help: did you mean to capture by reference instead?
= note: `#[warn(unused_assignments)]` (part of `#[warn(unused)]`) on by default

warning: value assigned to `contador` is never read
--> HelloWorld.rs:8:9
|
8 |         contador += 1;           // a thread altera a variável de main
|         ^^^^^^^^^^^^^^^
= help: maybe it is overwritten before being read?

warning: 2 warnings emitted

0
```

Saída do programa

não respondido

The screenshot shows a Rust IDE with a file named 'HelloWorld.rs'. The code defines a main function that creates a shared counter 'contador' protected by a mutex. It then spawns four threads, each of which increments the counter 100,000 times. However, the threads are created using 'move' semantics, which means each thread receives a copy of the 'contador' variable (which is 0) instead of a reference to the shared variable. The main thread prints the value of 'contador' after all spawned threads have finished, resulting in 0. The right-hand pane shows the console output: 'contador = 400000'.

```
1 // Prática 6b - Rust: compartilhar com Arc (contagem de referências) e Mutex (exclusão mútua)
2 // Cole em https://onecompiler.com/rust
3 use std::sync::{Arc, Mutex};
4 use std::thread;
5
6 fn main() {
7     let contador = Arc::new(Mutex::new(0)); // compartilhado e protegido
8     let mut threads = Vec::new();
9     for _ in 0..4 {
10         let c = Arc::clone(&contador);
11         threads.push(thread::spawn(move || {
12             for _ in 0..100_000 {
13                 *c.lock().unwrap() += 1; // trava, soma e destrava
14             }
15         }));
16     }
17     for t in threads { t.join().unwrap(); }
18     println!("contador = {}", *contador.lock().unwrap());
19 }
```

contador = 400000

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

O programa imprime 0 porque a palavra-chave move força a nova thread a receber a propriedade (ownership) das variáveis que ela usa.

Como contador é um número inteiro simples (um tipo que implementa o comportamento de Copy), o move realiza uma cópia idêntica do valor 0 para dentro da thread.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em que momento o erro foi descoberto?

- Java (Prática 1): O erro foi descoberto em tempo de execução (runtime). O código compilou perfeitamente e sem avisos, mas apresentou inconsistências matemáticas e perda de dados de forma imprevisível enquanto o programa rodava.
- Rust (Prática 6a): O erro foi descoberto em tempo de compilação (compile-time). O compilador do Rust identificou o risco de segurança na memória através do seu sistema de Borrow Checker e se recusou a gerar o executável.

Qual é a vantagem e qual é o custo para quem programa?

- Vantagem: A segurança é garantida antes que o código vá para a produção. Em Rust, se o código compilar, o programador tem a certeza matemática de que ele é livre de condições de corrida de dados (data race free). Isso elimina bugs fantasmas que só acontecem sob alta carga no cliente final.
- Custo: A curva de aprendizado inicial é muito mais íngreme. O programador precisa passar mais tempo "lutando com o compilador" para arquitetar corretamente o compartilhamento de dados (escrevendo códigos mais verbosos como o uso explícito de `Arc::clone` e `.lock().unwrap()`), o que pode desacelerar o desenvolvimento nas fases iniciais do projeto.

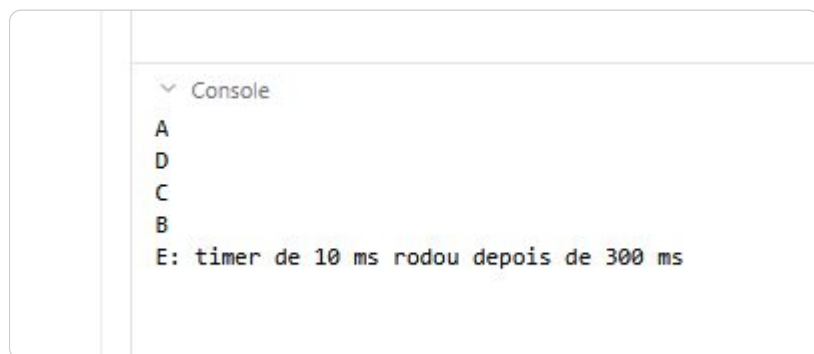
Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A, D, C, B

Saída do programa

não respondido



```
▼ Console
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

Por que B saiu depois de D?

O JavaScript executa o código síncrono imediatamente na linha de execução principal (Call Stack). O `setTimeout(..., 0)` é uma operação assíncrona enviada para a fila de tarefas do laço de eventos (Event Loop). Por isso, o programa primeiro executa as linhas puramente síncronas (`console.log("A")` e `console.log("D")`) antes de sequer olhar para a fila de eventos, empurrando a letra B para depois de D.

Por que B saiu depois de C?

O laço de eventos do JavaScript possui diferentes níveis de prioridade em suas filas de tarefas:

- Microtasks (Promessas): Possuem prioridade máxima. A fila de microtarefas (onde o `Promise.resolve().then(...)` joga a letra C) é processada inteiramente assim que a pilha síncrona esvazia.
- Macrotasks (Timers): Possuem menor prioridade. O `setTimeout` (onde estava a letra B) só ganha a vez de executar quando todas as microtarefas pendentes tiverem acabado.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

Por que rodou bem depois do tempo esperado?

O tempo definido em um `setTimeout` indica apenas o tempo mínimo de espera para que a tarefa entre na fila, e não o momento exato de sua execução.

A função `ocupado(300)` executa um laço `while` síncrono que consome 100% da única thread do JavaScript por 300 milissegundos. Como o laço de eventos só consegue processar a próxima tarefa quando a thread principal estiver totalmente livre, o timer de 10 ms ficou "preso na fila" aguardando o término dos 300 ms da computação pesada.

O que isso significa para um servidor Node.js?

Significa que o servidor ficará totalmente congelado para todos os usuários conectados.

Como o Node.js roda sob um modelo de thread única (Single Threaded), se uma requisição de um único usuário disparar um processamento pesado, síncrono e demorado (como criptografia pesada, manipulação gigante de JSON ou laços matemáticos complexos), essa thread única ficará bloqueada.

Nenhum outro usuário conseguirá receber respostas, e novos acessos ou tarefas agendadas ficarão parados na fila gerando atrasos em cascata, destruindo a escalabilidade do sistema. Operações pesadas de CPU no Node.js devem ser delegadas para bancos de dados ou processos paralelos (Worker Threads).

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo.

Qual problema desta atividade pode acontecer? Como você o evitaria?

Qual problema desta atividade pode acontecer?

Pode acontecer uma Condição de Corrida (Race Condition) idêntica à que vimos na Prática 1. O sistema pode processar as duas compras simultaneamente seguindo estes passos:

1. A transação da Pessoa A lê o banco de dados e vê: 1 lugar restante.
2. Exatamente na mesma fração de milissegundo, a transação da Pessoa B lê o banco de dados e vê: 1 lugar restante.
3. O sistema aprova a compra da Pessoa A (restam 0).
4. O sistema aprova a compra da Pessoa B (restam 0 ou -1).

O resultado prático seria o famoso Overbooking (venda duplicada do mesmo assento), gerando um erro grave no sistema.

Como você o evitaria?

A nível de banco de dados e aplicação real, isso é evitado através de Bloqueio Pessimista (Pessimistic Locking) ou Transações Isoladas:

- Bloqueio no Banco de Dados (SELECT ... FOR UPDATE): Quando a Pessoa A clica em comprar, o sistema bloqueia aquela linha específica do banco de dados correspondente ao ingresso. Se a Pessoa B tentar clicar no mesmo instante, o sistema obriga a transação B a esperar (ficar bloqueada) até que a transação A termine. Ao terminar, o saldo vira 0, e a transação B ao acordar verá que não há mais ingressos, cancelando a operação de forma segura.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

o formato da atividade (excepcional), a qualidade do conteúdo e a facilidade do entendimento.